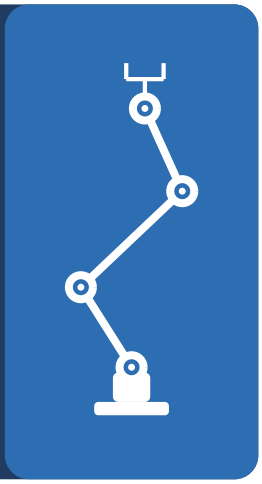
UR5e AI Motion Planner

Natural Language Robot Programming, Deterministic Motion Validation, Kinematic Planning and Safety Architecture

VERSION 2.1 · 03 NOVEMBER 2026 · ENGINEERING REPORT



Author / Xamdamov Abu Bakr

UR5e · Python · Ollama / gpt-oss:20b · URScript · RTDE · OnRobot

Abstract. This report documents a local AI-assisted robot programming system that converts operator language into structured motion plans. Python computes geometry, checks a constrained instruction language and validates sampled paths against workspace and kinematic conditions. Task approval and fresh-state checks precede dispatch; telemetry distinguishes confirmed outcomes from unconfirmed transmission. The report preserves the original architectural swimlane, expands the mathematical and implementation detail, and reconciles historical claims with the supplied source. Physical testing evidence is explicitly separated from offline results.

Contents

1 Introduction and project scope 2
2 Complete system architecture 3
3 End-to-end flow and bounded repair 4
4 Local model integration and plan example 6
5 Deterministic resolution and geometry 8
6 Coordinate frames and kinematics 10
7 Deterministic validation and path sampling 12
8 Reach envelope and target adjustment 14
9 URScript program generation 16
10 Human approval and execution integrity 17
11 Robot communication and telemetry 18
12 Execution and confirmation 19
13 Independent stop and fault handling 20
14 Built-in workspace corner tour 21
15 Software modules and terminal behavior 22
16 Result states 23
17 Offline and physical verification 24
18 Limitations, improvements and conclusion 26
A Configuration reference 27
B Source reconciliation 28
C References and traceability 29

Introduction and project scope

The UR5e AI Motion Planner translates operator language into a constrained motion program for a Universal Robots UR5e. It combines local language interpretation with deterministic coordinate computation, program validation, operator review and telemetry-based execution reporting. The implementation is a terminal application, not a vision system, a general autonomous manipulation agent or a replacement for the controller trajectory planner.

The governing architecture

The AI proposes. Deterministic software validates. The human approves the task. The controller executes. Telemetry provides evidence of completion. Each clause describes a different responsibility. A plausible sentence from a model is not a motion command; a syntactically correct instruction is not proof of reachability; and a successful socket write is not proof of physical arrival.

The supplied source supports Cartesian lines, explicit joint moves, circular arcs, generated polygons and circles, bounded repetition, gripper commands, waits, saved poses and a built-in workspace tour. Numeric geometry is computed in Python. The robot controller remains responsible for actual motion execution. This report documents the supplied seven Python modules, rather than assuming that every feature described in an earlier PDF is present in this snapshot.

Evidence and version boundary

Table 1. Introduction and project scope

Evidence class	What it establishes
Source-verified	An implemented branch, parameter, formula or interface in the attached Python files. This is not a physical performance measurement.
Historically reported	The earlier AI report states 83/84 offline checks. Its test scripts and raw outputs were not attached, and several described features differ from the source.
Setup-specific report	The README describes real-controller HOME data and an OnRobot integration used on a tested setup. The project author confirms that all physical hardware tests passed.
New offline checks	A small reproducible, network-blocked inspection accompanies this report. Results and scope appear in Section 17.

What is deliberately not claimed

The software uses sampled, approximate geometry. It does not establish complete obstacle clearance, certified stopping performance or guaranteed natural-language understanding. The separate Gesture Control PDF supplies the publication design reference; its physical-test results belong to a different application and are not transferred to this one.

> **Implementation exception:** startup homing and the direct /home command call Robot.home() without the task APPROVE dialog. The approval principle applies to the AI-task and /robot paths; it is not universal in the supplied code. See Section 10.

Source basis: main.py, Robot.home(), resolve_plan(), README.md and the original AI report. Uploaded filename suffixes are omitted in code references for readability; Appendix B identifies the exact files.

Complete system architecture

Figure 1 separates untrusted interpretation, deterministic computation and controller execution. The terminal owns interaction; ai_agent.py coordinates candidate construction; robot_sender.py owns geometry, validation and transport. The same validator checks generated AI tasks and deterministic tour plans, avoiding a separate permissive path for the tour.

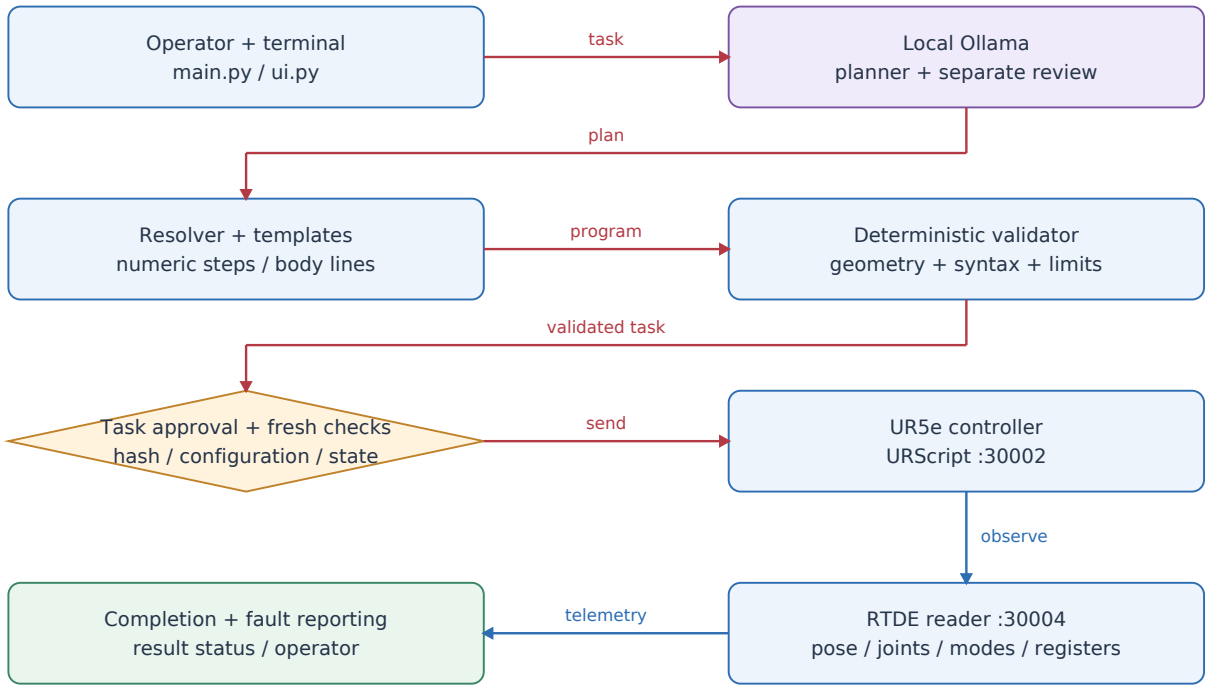


Figure 1. Command and observation responsibilities. Red arrows carry task/program flow; blue arrows carry state or feedback.

Trust boundaries

Ollama is contacted through localhost:11434 using HTTP. Its response selects from a schema; it cannot open a robot socket or invoke arbitrary Python through the exposed planner tools. The only planner tools are get_robot_state and get_limits. The instruction parser accepts a narrow grammar instead of forwarding arbitrary model-generated URScript.

A Candidate contains the task, raw plan, resolved steps, generated code, findings, review and approval fingerprint. A ValidationResult carries normalized body lines, their SHA-256, estimated duration, starting pose and predicted endpoint information. Robot.execute() consumes this result and independently repeats checks before transmission.

Controller boundary

URScript travels to port 30002; RTDE observations return through port 30004. The Dashboard connection on 29999 provides status queries and stop. These are independent software channels, not independent safety-rated systems. The application does not power on the arm, release brakes or clear protective stops. [R1, R2]

> The diagram is a responsibility map. The exact candidate order is resolve → render → validate → review. The preserved historical swimlane on the next page places review before validation; Section 3 explains this difference.

End-to-end control flow

The four-lane flowchart is preserved as vector artwork from page 2 of the supplied AI PDF. Its operator, AI, software-core and controller lanes retain the original decisions, red control arrows, approval branches and repair loop.

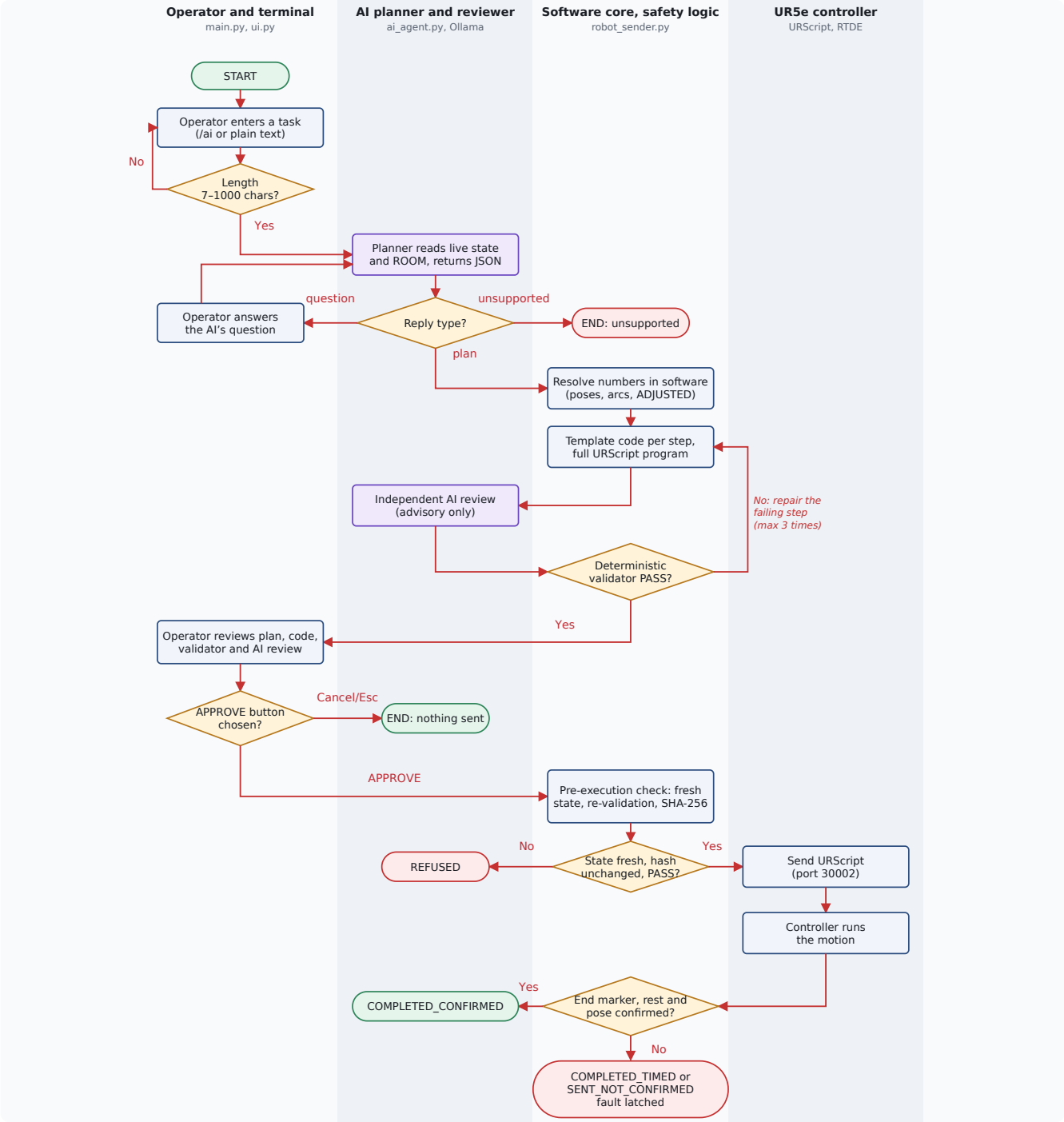


Figure 2. Preserved original end-to-end swimlane. Historical schematic, not a substitute for the source-specific execution semantics on pages 5, 17 and 19. In this snapshot validation precedes review; pose-only confirmation exists when the marker is unavailable; direct homing bypasses the task approval dialog.

Pipeline, findings and bounded repair

A task begins with 7–1000 characters of operator text. Agent.begin_task() saves the task-start pose separately from the application session-start pose. Clarification answers become part of the task context, so a later review can compare the final program against both the original request and the answers. The planner may ask for state, ask a question, declare the request unsupported or return a plan.

Actual candidate construction

Agent.build_candidate() resolves a deep copy of the plan against a current state snapshot. Templates generate one instruction per resolved step. A signature of action and parameters lets unchanged step code be reused on subsequent attempts. validate_candidate() checks both step/code agreement and the complete assembled program. Only a passing deterministic result proceeds to a separate review call with fresh context.

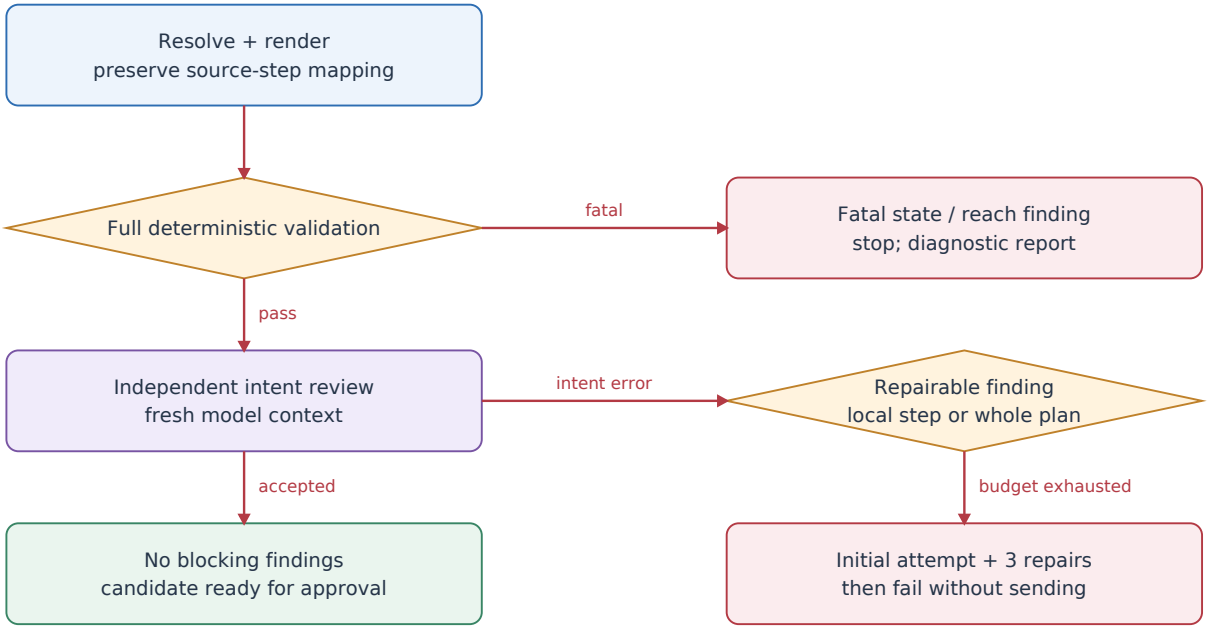


Figure 3. Candidate lifecycle and repair policy. Every repair returns to full resolution and validation.

Why repairs are local

A bad early position can cause several later geometric failures. The repair selector therefore changes the first dependent geometric failure, while independently invalid speed, acceleration, joint, sleep and gripper fields can be repaired separately. Findings carry source paths into the original plan, including repeat-body paths. The full pose chain is rebuilt afterward; later failures are never simply ignored.

Fatal configuration/state findings and fit or reach findings are not delegated to the model for arbitrary correction. Template-mode code mismatch is treated as an internal failure. Global findings may trigger a minimally changed whole-plan request. Repeated identical rejected plans or step revisions terminate the attempt. JSON retries, empty-response retries and the three repair attempts are separate budgets.

> “Independent review” means a separate call and context, using the same configured model. It is not independent software assurance. Reviewer errors can block a candidate; an exact-start precision objection can be overruled by deterministic distance facts.

Local model integration

The original physical implementation was tested with **gpt-oss:20b**. OLLAMA_MODEL in limits.py can select another installed Ollama-compatible model, but its performance and reliability have not been verified. check_ollama() requires the configured exact tag; the application does not pull or silently substitute models. The tested default endpoint is http://localhost:11434.

Table 2. Local model integration

Setting	Configured value	Effect
API	POST /api/chat; stream=false	One complete response is processed per request.
Temperature / context	0.1 / 16,384 tokens	Low sampling temperature; requested context budget.
Output / thinking	8,192 tokens / medium	Reasoning may consume part of the response budget.
Request timeout	600 seconds	Long local inference can fail with an explicit error.
JSON / empty retries	2 / 2	Invalid schema and empty content have bounded retries.
Planner tool calls	Maximum 3	Only state and limit observations.
Code mode	template	Python renders canonical instruction lines.

Prompt construction

prompts.py reads active capabilities, numeric limits and the direction dictionary at call time. state_summary() supplies TCP position, rotation vector, joint angles, modes, speed, saved pose references and a directional ROOM report. ROOM is model-derived available travel, not measured free space around external obstacles. Directions are expressed in the robot base frame: up +Z, down −Z, forward +X, backward −X, left +Y and right −Y in this configuration.

The HTTP payload includes model, messages, options and think. Although Ollama offers a format field for structured output, this implementation relies on text prompting followed by local JSON extraction and schema checking; it does not set format="json". That distinction matters when diagnosing malformed responses. The returned model name is checked when present. [R4]

Response handling and limitations

A valid planner response has action tool, clarify, unsupported or plan. A plan includes interpretation, operations, assumptions, steps and three boolean self_check fields. All self_check values must be true to pass the outer schema, but a model declaring itself correct is only an input check. Step fields and numeric feasibility are checked later by deterministic code.

Unsupported requests include conditional loops, sensor-driven decisions, camera interpretation and unrestricted URScript. Missing object locations should produce clarification. Prompt instructions discourage invented coordinates, but the validator cannot establish whether a numerically valid coordinate corresponds to the user’s intended physical object.

> The stop-input thread remains separate from blocking HTTP inference. A stop request can be dispatched without waiting for the model; the inference request itself may still run until its response or timeout.

Structured plan and worked example

Example request: “Move 20 mm up, then return to the exact starting pose.” The following is an implementation-compatible illustrative plan, not a recorded model response or a physical-test log. It uses a delta and a saved reference so the model does not invent absolute coordinates.

```
{
  "action": "plan",
  "interpretation": "Lift 20 mm along base +Z and return.",
  "operations": ["movel"],
  "assumptions": ["Up means base +Z."],
  "steps": [
    {"name": "Lift", "action": "movel",
     "description": "Lift 0.020 m (20 mm).",
     "params": {"delta_xyz_m": [0, 0, 0.020]}},
    {"name": "Return", "action": "movel",
     "description": "Return to the saved task-start pose.",
     "params": {"pose_ref": "start"}}
  ],
  "self_check": {"covers_all_requested_actions": true,
                 "end_matches_request": true,
                 "no_extra_steps": true}
}
```

Resolution from the configured HOME example

For an illustrative start at HOME, p_0 = (0.492460, −0.135581, 0.260306) m. The requested translation is Δp = (0, 0, 0.020) m, giving p_1 = (0.492460, −0.135581, 0.280306) m. The rotation vector remains (2.217481, 2.218513, −0.007947) rad. The return step copies the saved full six-component pose, rather than subtracting a rounded displacement.

```
movel(p[0.49246, -0.135581, 0.280306,
        2.217481, 2.218513, -0.007947], a=0.05, v=0.02)
movel(p[0.49246, -0.135581, 0.260306,
        2.217481, 2.218513, -0.007947], a=0.05, v=0.02)
```

The displayed line wraps are for publication only: each canonical instruction is emitted on one line. Defaults are 0.02 m/s and 0.05 m/s²; the caps are 0.05 m/s and 0.1 m/s². A valid arithmetic example still needs the real initial state, tool model and complete path validation before execution.

Two meanings of “exact”

The deterministic review fact compares the requested return position with the saved start using a 0.0001 m tolerance. Physical motion confirmation uses a looser 0.002 m positional tolerance and 0.02 rad orientation tolerance. Therefore exact plan reconstruction does not mean zero physical positioning error. The task reference start is captured per task; session_start is captured before startup homing.

Deterministic motion resolution

resolve_plan() advances a numerical pose chain, expands compound operations and attaches source-step metadata to every generated instruction. Numeric inputs must be finite; booleans are not accepted as ordinary numeric values. Cartesian units are metres, joint and rotation-vector units are radians, and gripper fields explicitly use millimetres and newtons.

Table 3. Deterministic motion resolution

Primitive / mode	Implemented meaning and restrictions
movel delta_xyz_m	Add a three-component base-frame translation; retain orientation. This is relative to the current TCP position, not a tool-local vector transform.
movel pose	A six-component absolute pose. Optional adjust=true invokes the adjustment heuristic. xyz_m specifies absolute position while retaining tracked orientation.
movel pose_ref	Copy start or session_start. The code also accepts adjust=true here; avoid claiming saved references are unconditionally immutable.
movel rotate	Left-compose a rotation about base X, Y or Z; keep TCP position. angle_deg is converted to radians.
move / until_limit	Resolve a configured base-axis direction. Explicit distances do not silently shrink unless stop_at_limit is requested.
to_limits	Choose one or more workspace faces 1 mm inside the box, then attempt reach adjustment.
movej	Explicit joints_rad or base_rotate {angle_deg,via}. Requires known joints before Cartesian motion; validates joint and TCP paths.
movec	Via/end poses or a geometric arc description; circle geometry and arc path are checked.
polygon / circle	Software-expanded geometry: 3–12 polygon sides; two half-circle arcs for a full circle.
repeat / sleep	Expand 1–20 repeats; nested repeats and shapes inside repeats are rejected. Each sleep must be >0 and ≤10 s.
home / gripper	HOME becomes a movel; gripper carries width_mm and force_n.

Intent and geometry remain separate

A line move requires exactly one positioning mode. A delta greater than 1 m in any component is rejected as a likely unit error. A direction-based distance requires exactly one distance field unless until_limit is true. Unknown direction words are refused instead of being mapped from camera or operator orientation.

The resolver limits the raw plan to 15 steps and the expanded program to 40 lines, recording a finding when truncation occurs; a truncated plan is not accepted as a complete task. Shapes are expanded at the top level, then checked by the same instruction validator. All resulting numbers are inspectable before the task approval stage.

> **base_rotate** is a movej parameter: angle_deg in (0,360] and via +Y or −Y. Only joint 1 turns; 37 FK samples select the requested swing side. It requires known live joints/tool before Cartesian motion. For radius r=max(TCP radial distance,0.05 m), omitted speed/acceleration are capped at min(default,0.9 v_max/r) and min(default,0.8 a_max/r). Explicit values still undergo full validation.

Geometric construction and timing

The geometry layer uses vectors and matrices implemented with Python’s standard library. Figure 4 shows why the model selects geometric intent while software computes waypoints. Validation works on the expanded result, so an incorrect shape cannot bypass checks by appearing as a high-level instruction.

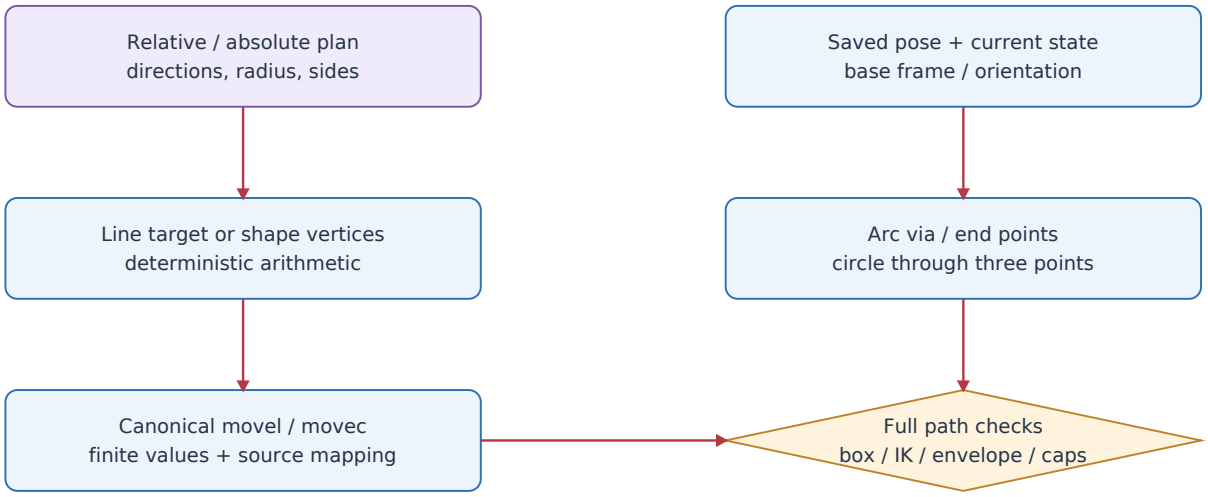


Figure 4. Geometry expansion. Every generated primitive joins the same canonical-code and validation path.

Arc construction

An arc uses a selected plane, centre direction, turning direction, radius and sweep. With unit in-plane vectors u and v, a point on the circle is $p(\varphi) = c + r[u \cos \varphi + v \sin \varphi]$. The start, via and end positions define a circle; collinear or nearly coincident points are rejected. The via point selects the intended sweep rather than merely supplying an endpoint.

Allowed arc radii are 0.005–0.5 m and a single sweep cannot exceed 350°. A full circle is expanded into two 180° movec segments. The first half receives a 0.001 m blend and the final half stops at its endpoint. The validator limits a blend to 25% of the shorter adjacent waypoint spacing and rejects consecutive movec instructions when the preceding blend is zero. These are implementation rules, not a proof of the controller’s blended swept volume.

Polygon and duration calculations

For a regular n-gon, the central angular increment is $2\pi/n$. expand_polygon() derives vertices from the current point and selected plane; the final segment returns to the initial vertex. The model is instructed not to enumerate rounded corners itself. A repeat is a software expansion, not a controller loop with runtime conditions.

For translational length d, speed v and acceleration a, the triangular/trapezoidal estimate is $t = 2\sqrt{(d/a)}$ when $d < v^2/a$, otherwise $t = d/v + v/a$. _movel_time() also estimates rotation and uses the larger duration. This is a timeout estimate, not measured cycle time. execute() allows twice the estimated program duration plus 15 seconds.

Coordinate frames and forward kinematics

A TCP pose is [x, y, z, rx, ry, rz]. Position is expressed in the base frame; the last three values form an axis-angle rotation vector, not roll, pitch and yaw. Its norm is the rotation angle and its normalized direction is the rotation axis. Joint space is q = [q_1,...,q_6], measured in radians.

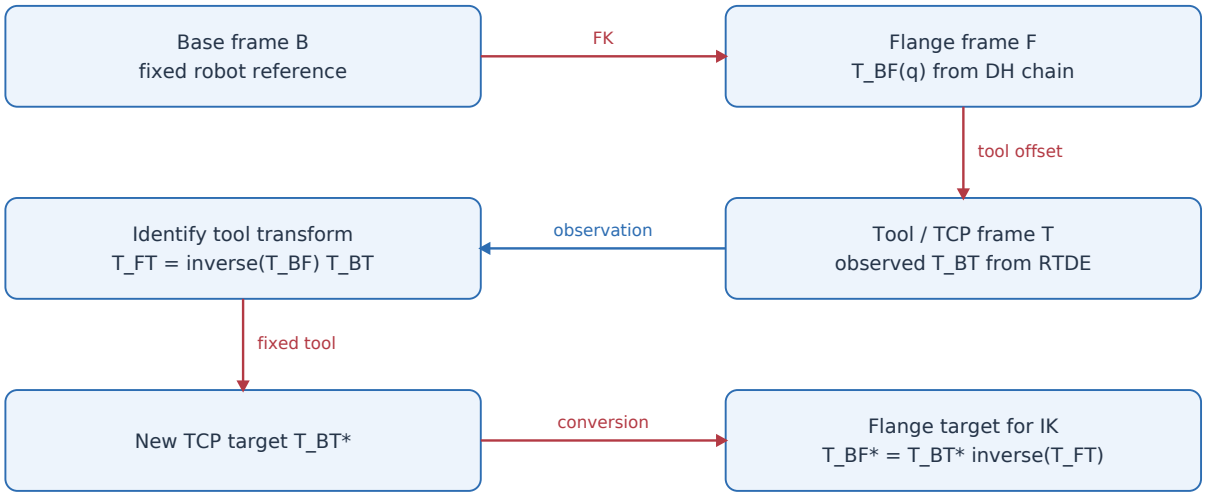


Figure 5. Frame composition and tool identification. The nominal flange model and observed TCP pose determine the fixed tool transform.

Nominal Denavit–Hartenberg chain

fk_flange() multiplies six standard DH transforms: T_BF(q) = $A_1(q_1)$ $A_2(q_2)$ … $A_6(q_6)$. Each transform uses the joint angle, link length a, link offset d and twist α. The code uses the following nominal UR5e parameters, consistent with the manufacturer's published kinematic dimensions. Individual robot calibration is not loaded. [R3]

Table 4. Coordinate frames and forward kinematics

Joint	a (m)	d (m)	α (rad)
1	0	0.1625	π/2
2	−0.425	0	0
3	−0.3922	0	0
4	0	0.1333	π/2
5	0	0.0997	−π/2
6	0	0.0996	0

Tool offset and orientation

The translational tool offset is o_F = R_BF^T(p_BT − p_BF). ToolModel also derives R_FT = R_BF^TR_BT. A target flange rotation is R_BF* = R_BT*R_FT^T and its position is p_BF* = p_BT* − R_BF*o_F. Offsets longer than 0.6 m are considered implausible by tool_from_state().

> After movej, the current resolver and validator recompute TCP orientation as R_BF(q)R_FT. If the tool orientation cannot be established, later TCP motion is refused. Use xyz_m for absolute position with the tracked orientation, especially after base_rotate. The earlier snapshot retained old orientation components; that limitation is historical.

Inverse kinematics and continuity

The local solver is a damped least-squares numerical IK routine seeded from the observed joints, then from the previous accepted sample. It validates a sampled path locally; it is not a global planner that searches all robot configurations or routes around obstacles. Failure to converge can mean an unreachable target, a poor seed or numerical difficulty.

Implemented update

```
e = [p_target - p_flange(q), rotvec(R_target R_current^T)]
J[:,j] = finite_difference(flange_pose, q[j], h=1e-6)
y = solve(J J^T + 1e-4 I, e)
dq = J^T y
if max(abs(dq)) > 0.3: dq *= 0.3 / max(abs(dq))
q = q + dq
```

Position residuals are in metres and rotational residuals are in radians. The code combines them directly; it does not introduce a task-space weighting matrix. The 6×6 system is solved by Gaussian elimination with partial pivoting. A pivot below 10^{-12} is treated as unsolvable. Up to 60 iterations are allowed for each sample.

Table 5. Inverse kinematics and continuity

Condition	Threshold	Interpretation
IK position residual	≤0.0005 m	Numerical convergence, not controller accuracy.
IK orientation residual	≤0.005 rad	Rotation error at the candidate flange pose.
Solver step	≤0.3 rad maximum component	Local update clipping, not a robot velocity limit.
Sample-to-sample jump	≤0.35 rad maximum component	Reject an apparent configuration jump.
Elbow / wrist 2	$\lvert\sin(q_3)\rvert$, $\lvert\sin(q_5)\rvert \geq \sin(0.2)$	Heuristic avoidance of folded/stretched elbow and wrist singularity.
Stretch fraction	≤0.90 (current; 0.95 historical)	Shoulder-to-wrist distance relative to link-length sum.

Orientation interpolation

The helper _slerp() computes a relative rotation vector $w = \log(R_1 R_0^T)$ and evaluates $R(t) = \exp(t[w]\times)R_0$. Rodrigues' formula maps a rotation vector into a matrix. rotation_angle_between() compares matrices using $\arccos(\text{clamp}((\text{trace}(R_0^T R_1)-1)/2))$, avoiding direct subtraction of axis-angle components.

reach_walk() follows samples in order and stops at the first failed IK or envelope condition. The preceding solution seeds the next solve. This encourages continuity but does not establish that the controller will follow the same numerical joint branch. The singularity checks are selected trigonometric heuristics, not a complete Jacobian condition-number assessment.

> Current reach sampling uses $N=\max(1,\text{ceil}(d/\Delta s),\text{ceil}(\theta/0.10-10^{-9}))$. straight_points() accepts endpoint rotations; reach_walk() further densifies insufficient lists. Pure in-place rotations therefore receive angular samples. The earlier translation-only implementation used one interval at d=0; that observation is historical.

Deterministic validation architecture

validate_candidate() first compares every step’s code with its expected primitive and numeric parameters, accepting at most 10^{-6} difference per numeric field. It then calls validate_body() for the assembled program. The latter has no execution side effects: it returns findings and predicted metadata. Repeated checks before sending use the same body validator.

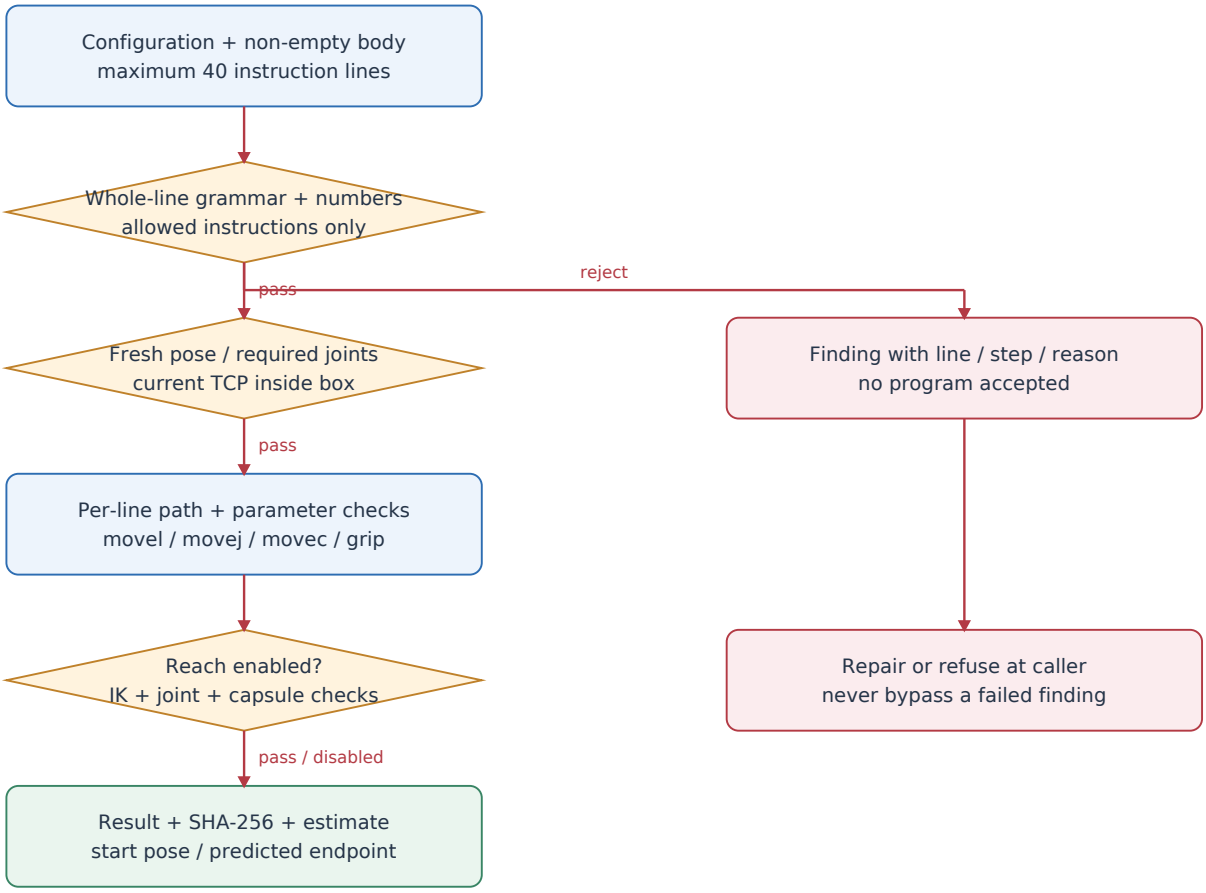


Figure 6. Validation includes always-on TCP base keep-out; disabling reach omits only the optional kinematic/capsule envelope.

Error classification and diagnostic value

A Finding can identify kind, source plan path, resolved step, instruction line, affected parameter, numerical limit and rejected instruction. This preserves the connection from an expanded repeat to the original user-visible step. Fatal findings describe configuration or state failures; code findings describe grammar or step mismatch; plan findings describe invalid motion or parameter choices.

The current TCP must be inside the configured box before any motion program is accepted. With REACH_CHECK_ENABLED true, the current joint/tool state must also satisfy the envelope. The software can therefore refuse even a command intended to recover from a disallowed starting state. That is the implementation’s refusal policy; it does not compute a recovery trajectory.

> The current validator always applies keepout_hit(), independently of REACH_CHECK_ENABLED. It rejects sampled TCP points with hypot(x,y)<0.20 m below z=0.60 m. A path starting inside may leave, but may not approach closer than its initial radius minus 10^{-6} m. Optional link/base capsule checking remains a separate envelope layer.

Path sampling and numerical checks

Table 6. Path sampling and numerical checks

Validation condition	Implementation / outcome
Cartesian box	Current X −0.550–0.657; Y −0.550–0.550; Z 0.020180–0.650 m. Earlier box: X 0.150–0.657, Y ±0.320, Z≤0.500 m.
Cartesian speed / acceleration	$0 < v \leq 0.05$ m/s; $0 < a \leq 0.1$ m/s² using active session caps.
Rotation vector	Norm must not exceed 2π rad for movel and arc via/end poses.
Joint position	q_1, q_4, q_5, q_6 in [−6,6]; q_2, q_3 in [−3.3,3.3] rad.
Joint speed / acceleration	$0 < v \leq 0.5$ rad/s; $0 < a \leq 1.0$ rad/s²; additional estimated TCP caps apply.
Workspace sampling	Straight lines and circular arcs: 0.005 m nominal spacing.
Reach sampling	Straight lines / arcs: 0.020 m; tour segments: 0.010 m.
Joint interpolation	201 TCP samples for workspace; every fourth sample for envelope geometry.
Gripper / wait	Width 3–106 mm; force 3–40 N; sleep >0 and ≤10 s.

Cartesian path checks

For a straight segment of length d, workspace sample count is $N = \max(1, \text{ceil}(d/0.005))$. Samples $p_k = p_0 + (k/N)(p_1 - p_0)$ include both endpoints. Reach sampling uses the same form with 0.020 m. The convex box makes endpoint inclusion sufficient for an ideal straight translation, but discrete sampling is also used for arcs and model geometry. No continuous swept-volume proof is performed.

Joint-space path and speed estimate

The movej checker linearly interpolates $q(s) = q_0 + s(q_1 - q_0)$, $s \in [0,1]$, at 201 points and applies forward kinematics with the inferred tool offset. Let $D = \max|q_1 - q_0|$ and $v_p = \min(v, \sqrt{(aD)})$. Finite differences approximate the largest $|dp/ds|$ and $|d^2p/ds^2|$. Estimated TCP peak speed is $\max|dp/ds| \cdot v_p/D$. Estimated acceleration adds $\max|dp/ds| \cdot a/D$ and $\max|d^2p/ds^2| \cdot (v_p/D)^2$.

These estimates can reject an apparently modest joint command because the TCP moves too quickly at a large lever arm. They approximate synchronized interpolation, not the exact controller trajectory. They are not a replacement for measuring speed and acceleration on the configured controller.

Boundary of the numerical model

Current sampling increases the count with tool rotation (maximum 0.10 rad per interval). Arc orientation interpolation still models start/end rotations rather than reproducing all controller blend behavior. Continuous swept-volume clearance, fixtures, cables and carried workpieces remain outside the sampled approximation.

Reach envelope and collision approximation

The allowed workspace is a rectangular TCP box; the arm’s feasible configurations form a different, orientation-dependent set. A point inside the box may require excessive extension, a singular configuration or contact with the base. Figure 7 illustrates the implemented tests rather than presenting a measured reach map.

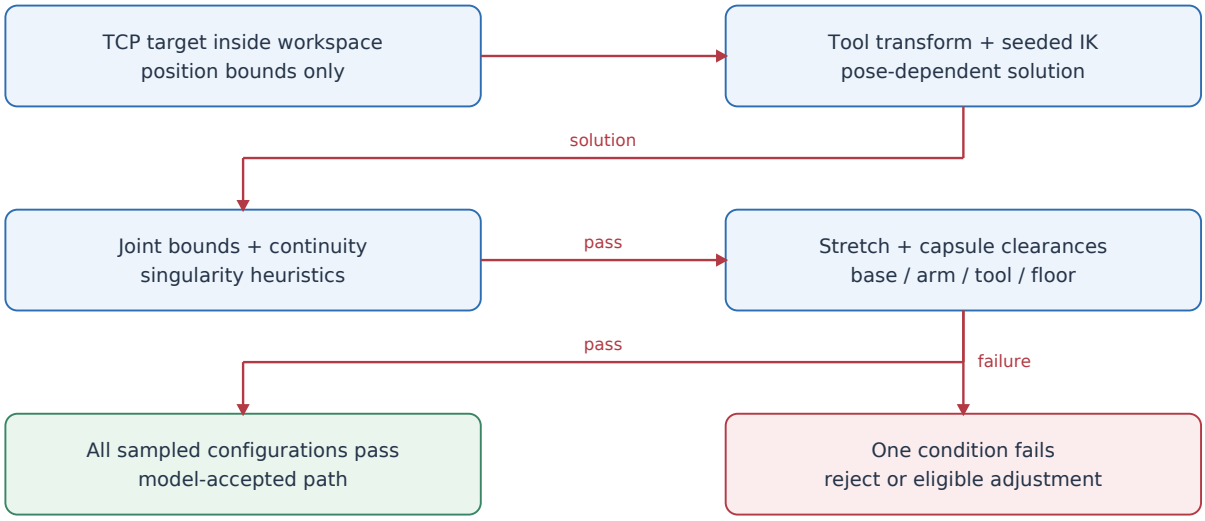


Figure 7. Reachability is an intersection of constraints, not a sphere or the workspace box alone.

Stretch criterion

The implementation computes $f = ||p_{wrist1} - p_{shoulder}|| / (|a_2| + |a_3|)$, denominator 0.425 + 0.3922 = 0.8172 m. Current acceptance is $f \leq 0.90$, or 0.73548 m shoulder-to-wrist distance. The earlier 0.95 cap and corresponding 0.77634 m ceiling are retained here as historical values. Neither ceiling is a maximum TCP reach; wrist geometry and tool length affect the TCP.

Capsule model

The base column is represented by a segment from (0,0,0) to (0,0,0.30) m with radius 0.10 m. Link radii are upper arm 0.08 m, forearm 0.07 m, wrists 0.06 m and tool 0.08 m. A 0.02 m margin is added between selected non-neighbouring capsule pairs. Neighbouring links are excluded because their simplified shapes naturally meet.

For tested segments A and B with radii r_A and r_B, clearance is $g = \text{distance}(\text{segment}_A, \text{segment}_B) - r_A - r_B - 0.02$. A negative g is rejected. The pair list includes column/forearm, column/wrists, column/tool, upper-arm/wrist2, upper-arm/wrist3, upper-arm/tool, forearm/wrist3 and forearm/tool.

The floor test compares the lower segment endpoint minus radius against FLOOR_Z_M plus margin. The tool is special-cased: its endpoint height is used without tool radius or extra floor margin, relying on TCP Z_MIN. This makes the model less conservative for tool/table contact than a full tool capsule against a plane.

> The approximate model knows neither fixture geometry nor cable routing, payload shape or grasped objects. README height figures are model-dependent estimates associated with a reported setup; they are not used here as measured performance data.

Automatic target adjustment

Adjustment changes the requested target and therefore must remain visible in the plan. The implementation emits an ADJUSTED note containing the original point, adjusted point and reason. It distinguishes a reach-envelope restriction from a workspace-face limit. It does not solve a global nearest-feasible-point optimization.

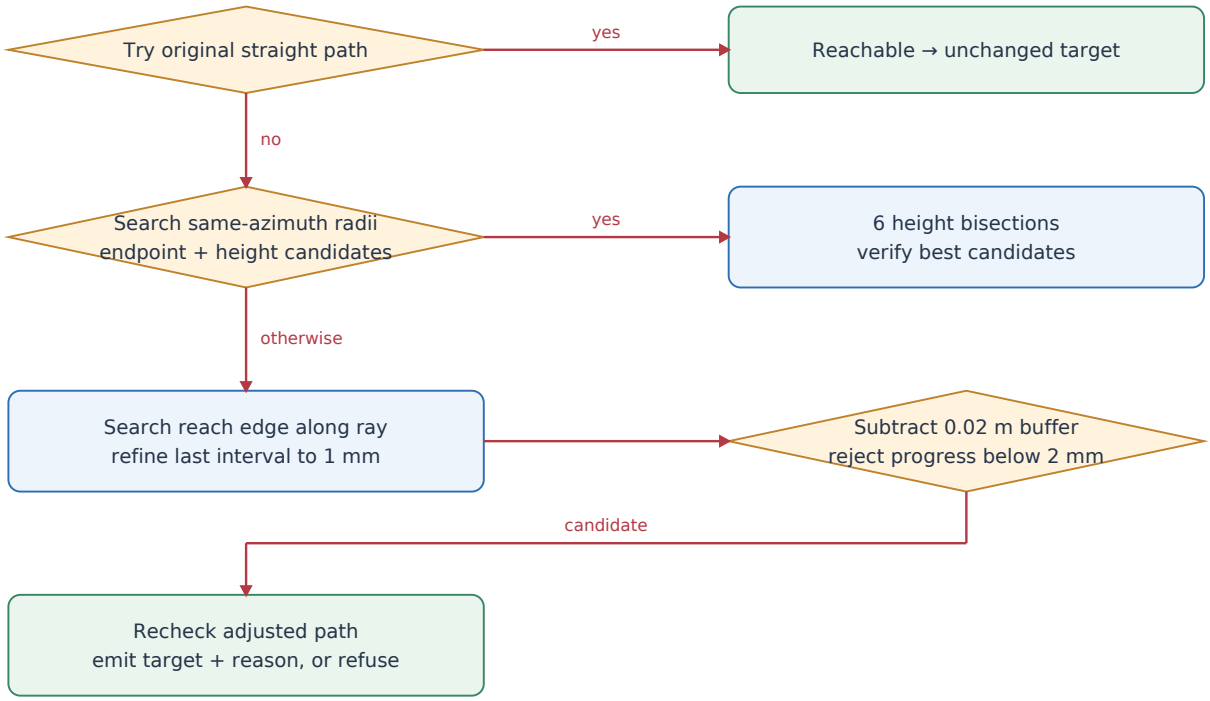


Figure 8. Current adjustment searches height/inward candidates and ray pullback; nearest verified candidate wins, with a 15% height preference.

Eligibility is mode-specific

An explicit pose or pose_ref uses this heuristic only when adjust is true. to_limits invokes it automatically when reach checking is active. A direction-based move first computes available room: until_limit uses that room, while an explicit distance is shortened only with stop_at_limit=true. A raw delta does not automatically run the height/pullback heuristic. HOME, arcs, shapes and joint commands do not have a general adjustment branch.

Height search and pullback

Current adjustment searches the target's vertical half-plane at the same azimuth. Radii decrease from the requested radius in 0.04 m steps down to max(keep-out radius+0.01,0.15) m. Endpoint screening tries current Z, the inset floor and their midpoint; six height bisections form candidates. The best few undergo full path and base keep-out checks. This replaces the earlier nine-bisection height-only branch.

The verified candidate nearest the requested point in 3D wins; unchanged-X/Y height adjustment is preferred if within 15% of the best distance. Candidates retreat toward the current position by the 0.02 m buffer and are rechecked. Ray pullback remains a fallback: reach-edge refinement is 1 mm, buffer 2 cm, minimum progress 2 mm. This is a finite heuristic, not a proof of globally optimal adjustment.

> The current tour delegates target adjustment to the shared resolver geometry. The old tour used eight height bisections and a **5 cm** top-corner buffer; those are historical parameters. The current general adjustment buffer remains **2 cm**. See Section 14 for orientation following and verified retracing.

URScript program generation

expected_line() is the default code generator. It renders one allowed primitive from a resolved step with numeric values formatted to nine decimal places. Optional LLM code generation is constrained to the same primitive and values; check_step_code_matches() parses and compares the result. Templates reduce syntax variability, while validation remains required.

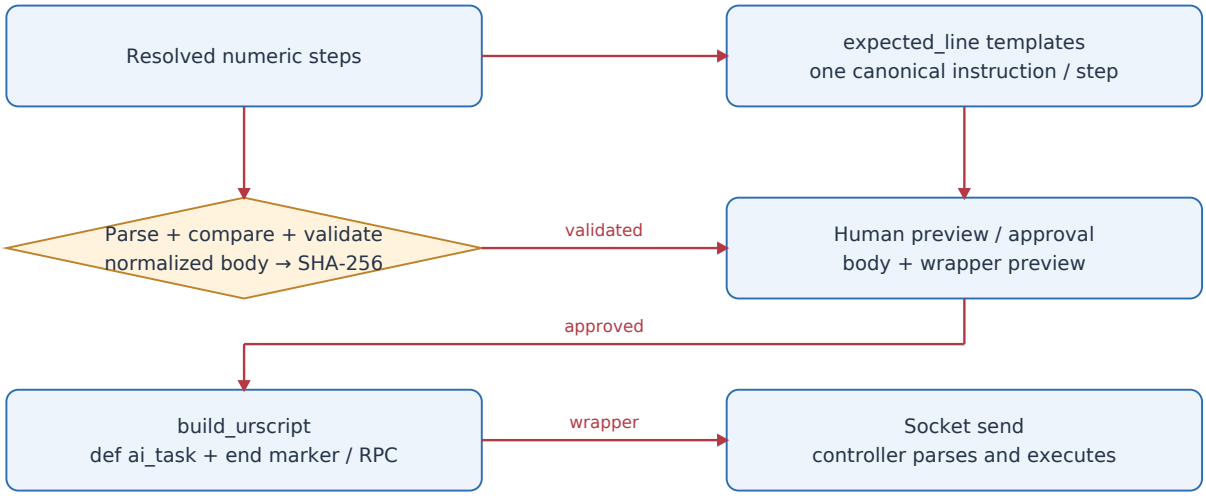


Figure 9. The approved body and the transport wrapper are distinct representations.

```
def ai_task():
  movel(p[0.49246, -0.135581, 0.280306,
          2.217481, 2.218513, -0.007947], a=0.05, v=0.02)
  write_output_integer_register(0, 12345)
end
ai_task()
```

The example uses a publication line wrap and an illustrative integer token. A real token is generated per dispatch. The hash covers the normalized validated body, joined with newline characters; it does not hash the final wrapper containing the runtime token. script_preview() uses the same wrapper builder and displays TOKEN in the saved approval record.

Gripper transformation

A validated rg_grip(width, force) is converted into an XML-RPC call to the OnRobot URCap service at http://localhost:41414 on the controller. The wrapper creates rpc_grip, calls rg_grip(0,width,force), checks its return code, shows an error popup and halts on nonzero return. It then sleeps for GRIPPER_MOVE_ESTIMATE_SECONDS, currently 3.0 seconds.

For a gripper-only program the wrapper writes a −1 sentinel to output float register 0, requests measured width with rg_get_width(0), and writes that value to the same register. It omits the arm-motion integer end marker. These service names and timing are setup-specific implementation assumptions; the supplied README reports successful use, but firmware/URCap version evidence is not attached.

> GRIPPER_SETTLE_SLEEP_SECONDS is configured as 3.5 s but is not the delay used by _urscript_body(); the wrapper uses the 3.0 s estimate. The report follows executable code rather than comments or unused configuration.

Human approval and execution integrity

AI tasks and /robot tours show the plan, resolved code, validator and review before execution. Interactive buttons default to CANCEL; arrows change selection, Enter confirms and Escape cancels. In a non-interactive terminal the exact word APPROVE is required. The approval timeout is 300 seconds.

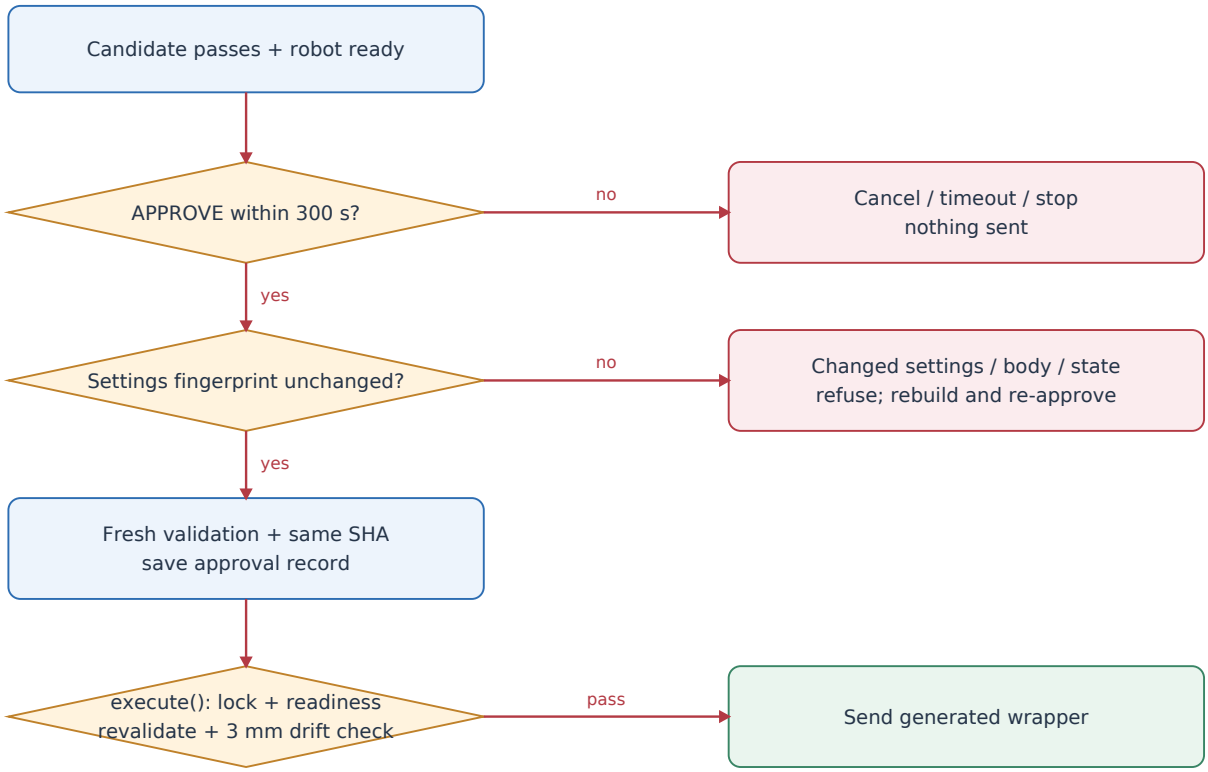


Figure 10. A passing candidate can still be refused. These checks apply to the task/tour approval path.

Fingerprint and fresh state

The configuration fingerprint contains active limits, LIVE lock state, HOME pose, model tag and gripper-enabled state. It is not a digest of every constant: capsule radii, reach thresholds and all timing parameters are not included. The body SHA-256 checks content equality, not origin authentication or a cryptographic signature from the operator.

present_and_execute() compares that configuration material, revalidates body lines with a fresh state and checks the hash. Robot.execute() repeats body hashing, takes a non-blocking busy lock, checks readiness and revalidates again. It rejects a starting TCP translation change greater than 0.003 m relative to the earlier result. There is no analogous explicit full joint/orientation drift comparison; fresh validation still uses the newly observed state.

Homing exception and audit scope

Robot.home() validates the HOME line and calls execute() using the result’s own hash. Startup invokes it when readiness passes; /home invokes it directly. Neither uses ask_approval(), the candidate configuration comparison or the approved-code report writer. This path is validated but does not implement the same explicit human approval contract as an AI task.

> approval/ records task, plan, step code, body and wrapper preview for the task/tour path. reports/ records failures and diagnostics. These files are ordinary local records, not a complete tamper-evident execution audit or proof that the controller executed a program.

Robot communication and telemetry

Robot communication is implemented with Python sockets and a compact RTDE reader. The application does not depend on an external rtde_receive package. Figure 11 distinguishes commands from observations; no live controller connection was made to produce this report.

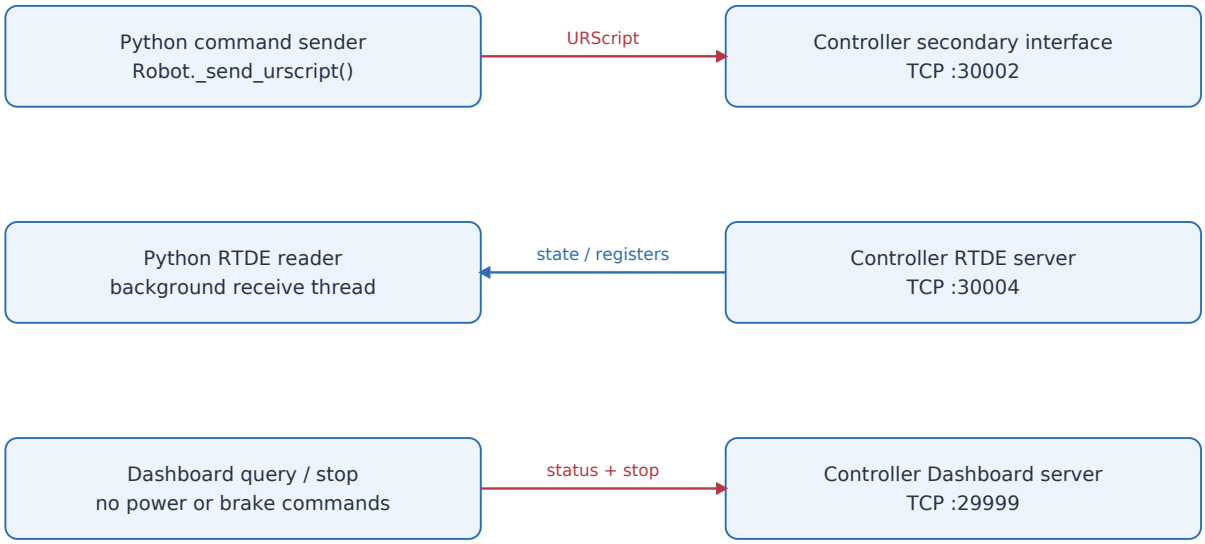


Figure 11. Network roles. The program sends URScript and stop requests, while RTDE carries observations back.

Connection and state data

The configured robot address is 192.168.X.X; it is a deployment setting, not a discovered device. Connect and read timeouts are 3 seconds. The reader requests RTDE protocol version 2 and output recipes at 10 Hz. Recipes include TCP pose/speed, joints/speeds, robot and safety modes, runtime state and available output registers. Reduced recipes accommodate missing fields or register availability.

RTDE state older than 1 second is normally unavailable. Execution monitoring requests a maximum age of at least 2 seconds; stop confirmation tightens it to 0.5 seconds. Polling at 50 ms does not create 20 Hz measurements from a 10 Hz recipe. A local timestamp indicates reception age, not a guaranteed end-to-end sensing latency.

Register semantics and controller compatibility

Motion wrappers use output integer register 0 as an end token; gripper-only wrappers use output float register 0 for measured width, observed as output_double_register_0. Manufacturer documentation reserves the lower output register range for Fieldbus/PLC use. The code negotiates availability, but deployment must establish that another component is not using those registers. [R1]

Port 30002 accepts URScript in the controller’s supported remote operating mode. A TCP send does not return a structured completion result. RTDE therefore supplies a separate observation path. Manufacturer guidance distinguishes e-Series remote and local modes; the relevant firmware and installation configuration must be recorded for repeatable physical testing. [R2]

> RTDE is used read-only at the application level: this client subscribes to outputs and does not configure input recipes for motion. The URScript wrapper itself writes output registers on the controller; those writes are not RTDE motion commands.

Execution and confirmation semantics

The execution lock permits one program at a time. Readiness requires a fresh state, robot_mode 7, safety_mode 1 or 2, no latched application fault and no runtime state PLAYING, PAUSING or RESUMING. The validator and LIVE switch are checked again. These are application gates, not a claim that all cell hazards have been assessed.

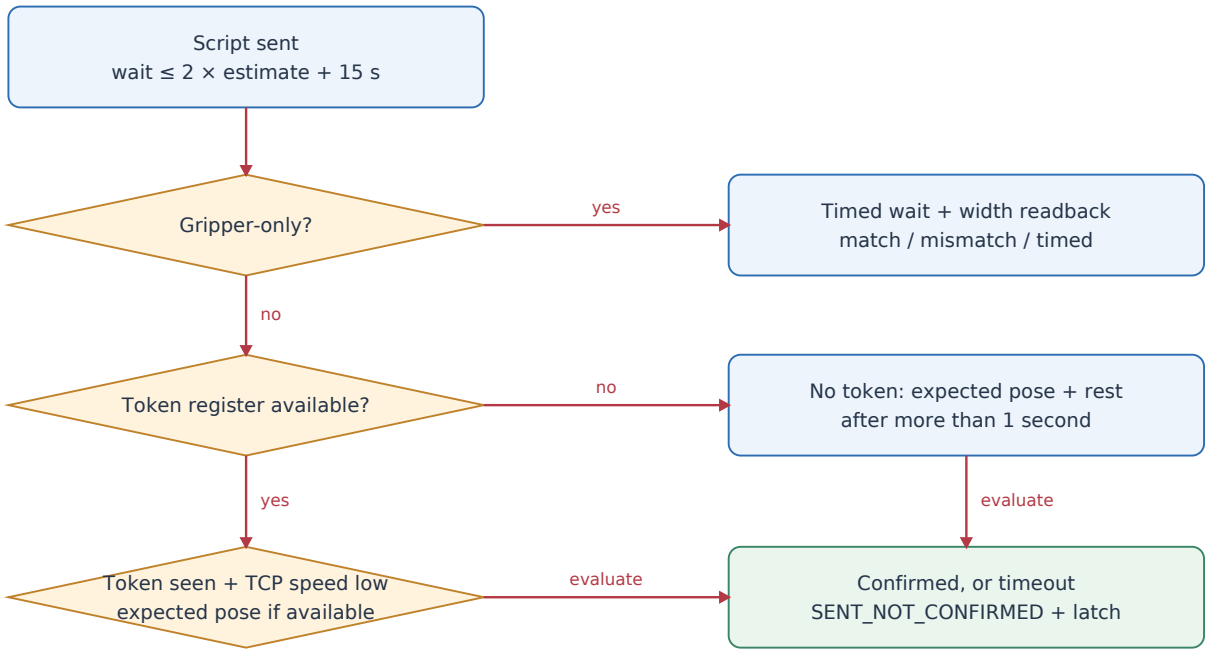


Figure 12. Actual completion branches. A no-marker fallback exists; final joint targets are not independently compared.

Motion path

With an available token register, the loop requires the dispatched token to have been observed, TCP speed below 0.002 m/s when that field is present, and expected pose agreement when applicable. Position tolerance is 0.002 m and rotation-angle tolerance is 0.02 rad. With no token register, an expected pose plus low speed after more than one second can also produce COMPLETED_CONFIRMED.

A final movej sets final_pose to None. The completion loop does not compare final_q with actual joints, so token and speed may be the only endpoint conditions for that case. Missing speed is treated as passing speed_ok; missing pose is not rejected inside the conditional pose comparison. The normal recipe supplies those fields, but the code’s conditional behavior must not be described as an unconditional proof.

Monitoring limitations

The arm completion loop does not continuously recheck safety_mode or robot_mode. Missing fresh telemetry is retried until timeout, rather than causing an immediate automatic stop in this function. A timeout returns SENT_NOT_CONFIRMED and latches a fault; that latch blocks later commands but is not itself a stop command. This differs from stronger guarantees stated in the older schematic.

Gripper-only execution waits for the estimated duration plus 2 s, checking stop requests and observed safety faults. It then waits up to 5 s for nonnegative width. A difference within 2 mm is confirmed; a larger difference produces COMPLETED_WIDTH_MISMATCH. Missing width produces COMPLETED_TIMED, which does not prove movement or grasp success.

Independent stop and fault handling

A separate console-input thread handles s during planning/execution without Enter. At the prompt, s, stop or /stop request the same software action. This bypasses the LLM, plan and approval.

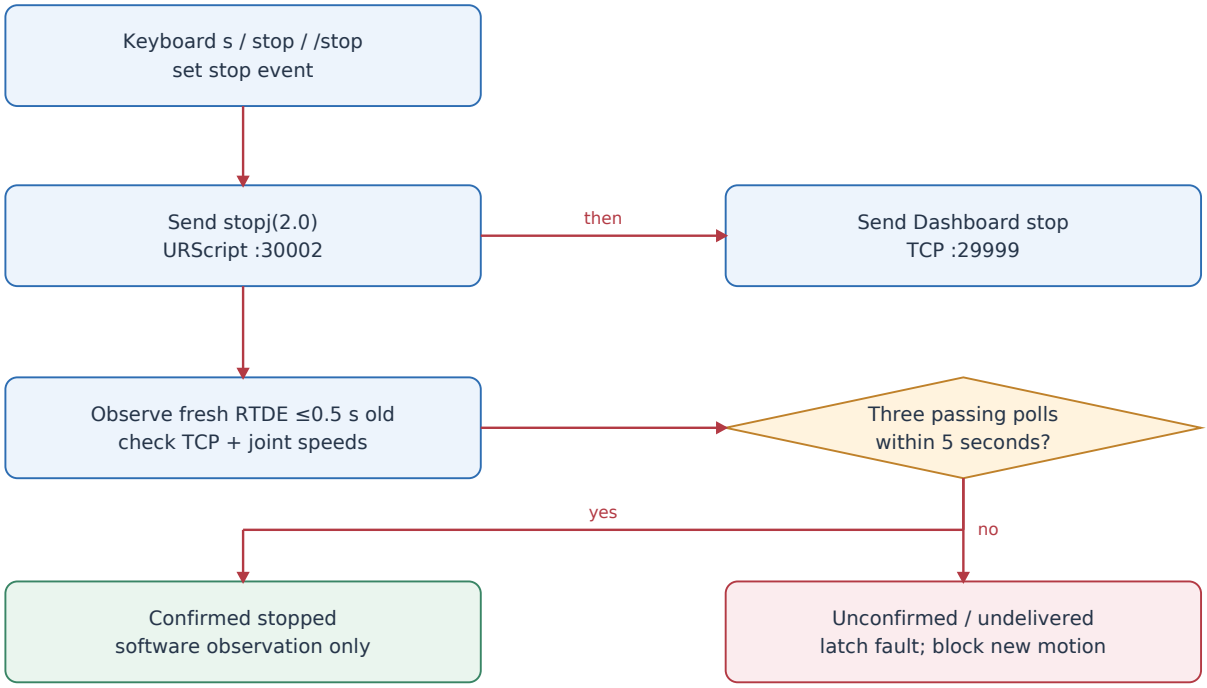


Figure 13. Stop request, delivery and observation are separate outcomes. Neither channel is a certified emergency stop.

Delivery is not confirmation

Robot.stop() sets an event, attempts stopj with 2.0 rad/s² deceleration, then attempts Dashboard stop even if the script channel fails. It returns whether any request was delivered and whether telemetry confirmed rest. Both channels share the controller and network, so their existence does not imply independent hardware redundancy.

_confirm_stopped() polls for up to 5 seconds. TCP translational speed must be below 0.002 m/s and maximum absolute joint speed below 0.005 rad/s where those measurements are available. Three consecutive passing polls yield true. If some velocity data was seen but rest was not established, the result is false; if no fresh usable state arrived, it is unknown. Polls are not required to carry distinct telemetry timestamps.

Fault lifetime and recovery

A failed or unconfirmed stop latches a reason string. Transmission errors and unconfirmed waited execution also latch faults. New motion readiness then fails. The supplied command interface has no implemented /clear-fault operation; /clear only resets task context. The execution-time message instructs verification of the robot followed by restarting the application. Restart clears application memory, but does not establish that the physical cell is safe or reset controller safety conditions.

STOPPED from the execution loop means the stop event was observed; the stop function’s separate confirmation result is the evidence about actual rest. A physical emergency stop, controller protective functions and external safeguarding remain distinct from these Python events and network messages.

> The /exit command ends the application without commanding a stop or a HOME move. Startup may later attempt validated homing.

Built-in workspace corner tour

The current syntax is /robot [speed] [stay] [fixed]; nohome aliases stay. By default tool yaw follows the arm azimuth; fixed selects the initial orientation. The tour is built without AI planning/review and uses the shared resolver, validator and task approval workflow.

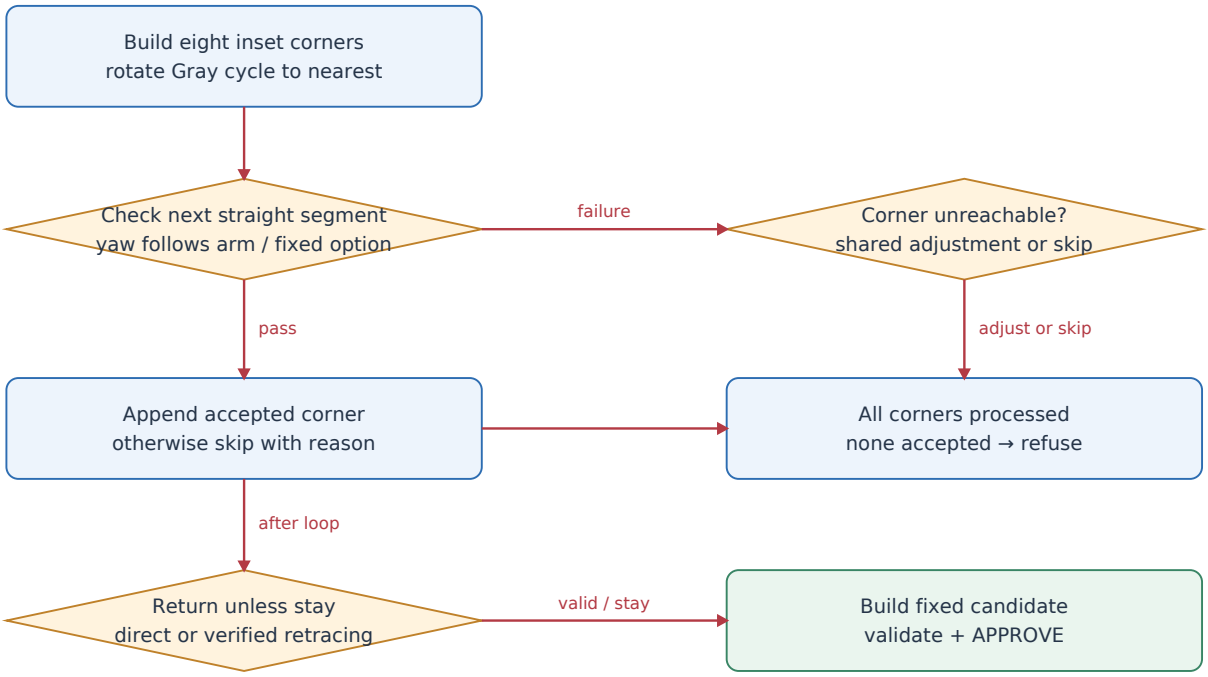


Figure 14. Current corner tour: yaw following or fixed orientation; direct return or re-verified retracing.

Corner geometry and order

Current inset corners are x∈{−0.549,0.656}, y∈{−0.549,0.549}, z∈{0.021180,0.649} m. Earlier values x∈{0.151,0.656}, y∈{−0.319,0.319}, z∈{0.021180,0.499} m are historical. The Gray cycle remains 000 → 100 → 110 → 010 → 011 → 111 → 101 → 001 → 000, rotated to the nearest starting corner; skipped corners can create diagonal connections.

Reach-limited corners and return

Tour segments use 1 cm translational sampling plus the rotation-step bound. Each target orientation follows arm azimuth unless fixed is selected. Unreachable corners are adjusted through the shared target heuristic, or skipped with a reason; no reachable corner means no candidate. Earlier eight-bisection/5 cm top-corner behavior is retained only as historical context.

Unless stay is requested, the tour checks a direct return to the saved start pose. If blocked, it walks backward through accepted corners, re-verifying each reverse segment and its changing orientation, until a direct return passes. If no verified return can be found it raises TourError. Retracing is therefore implemented, but never assumed valid without checking.

> Figure 14 documents implemented flow. Figure 17 in Appendix A shows the numbered Gray-code cube. Its coordinates describe ideal inset corners, not an assertion that all eight are physically reachable.

Software modules and terminal behavior

Table 7. Software modules and terminal behavior

File	Key entry points	Responsibility
main.py	startup; run_task; present_and_execute	Command dispatch, runtime checks, approval, audit-file writing and worker orchestration.
ui.py	Console; Pipeline	Input thread, terminal keys, buttons, history, completion, colors and progress log.
ai_agent.py	Agent; Candidate; build_candidate	Ollama calls, schema checks, clarification history, review and repair.
prompts.py	state_summary; planner_system_prompt	Capabilities, limits, task-state context and planner/reviewer prompts.
robot_sender.py	resolve_plan; validate_body; Robot	Geometry, kinematics, parser, templates, RTDE, execution and stop.
limits.py	get_limits; set_limit; check_consistency	Persistent defaults and session-only changes; LIVE lock and fingerprint material.
tour.py	build_tour_plan	Gray-code tour, shared reach adjustment, azimuth-following / fixed orientation and verified return retracing.

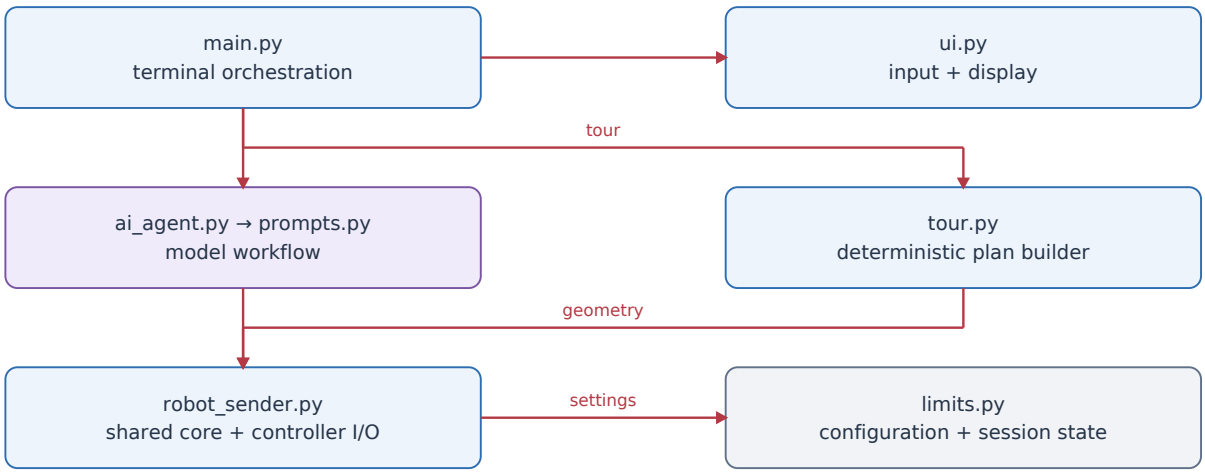


Figure 15. Module dependency map. Shared configuration and robot geometry underpin both task generators.

Threading and state

Console input and RTDE reception run separately from blocking task work. Pipeline draws an animated status line and can expand the process log. The execution busy lock prevents two dispatches from the same Robot object. A threading.Event communicates cancellation and stop requests. These mechanisms coordinate this process; they do not arbitrate other clients connected to the controller.

/limit updates an active session dictionary after finite-value, unit and consistency checks; it does not rewrite limits.py. /settings live off locks transmission without converting the application to simulation. --dry-run explicitly creates a simulated Robot and does not connect it to a controller. /clear forgets the task context but leaves hardware and settings unchanged.

> **Python 3.13+** is supported by the current startup version gate. The original physical implementation was tested using **Python 3.14.4**. Other supported Python versions have not all been tested. main.py checks MIN_PYTHON=(3,13); it does not enforce a fixed patch version.

Result states and operator interpretation

A status is a statement about the evidence collected by a particular branch. It should not be collapsed into a single “success” flag. In particular, accepted, transmitted, started, completed and independently observed are different events.

Table 8. Result states and operator interpretation

Status	Actual condition / practical meaning
COMPLETED_CONFIRMED	Arm: token plus low speed and expected pose when available; or no-token pose/rest fallback. Gripper-only: observed width within 2 mm. Does not prove grasp force, object identity or complete obstacle clearance.
COMPLETED_WIDTH_MISMATCH	Gripper width arrived but differs by more than tolerance. May reflect an object or a missed target; the software does not distinguish those causes.
COMPLETED_TIMED	Gripper-only timed wait elapsed, with no usable confirming width. Absence of a reported fault is not evidence of completed grip.
SENT_NOT_CONFIRMED	Waited motion timed out, or caller used wait=false. A timeout latches a fault; the immediate wait=false branch does not latch one.
STOPPED	Stop event observed during execution or the gripper wait. Consult the separate stop-delivery/rest result for physical confirmation.
REFUSED	Failed validation, execution lock, changed hash, busy state, readiness failure, fresh validation failure or excessive start translation drift. No dispatch in that branch.
FAILED	Socket transmission error or observed safety fault during the gripper timed wait; fault reason is latched. A socket failure can leave physical state unknown.
SIMULATED	Dry-run execution updates software state only. No controller script is sent.

Failure propagation

Planner errors are caught as LLMError and produce unsuccessful candidates. Fatal geometry/state findings terminate candidate construction; repairable findings can re-enter the bounded repair loop. Tour errors abort tour construction. None of these pre-execution failures is converted into a successful send. The terminal displays the result returned by Robot rather than inferring success from a lack of Python exceptions.

The wait=false branch deserves special attention when the Robot API is reused outside main.py. It reports SENT_NOT_CONFIRMED immediately after transmission, releases the local busy lock in finally and does not latch an unconfirmed-completion fault. The ordinary CLI uses the waited path, but a future integration must not assume that the status name alone implies a persistent lockout.

Evidence to retain after a physical run

Retain the exact body and wrapper token, initial state, command-send time, result status, terminal stop output and final RTDE samples. A screenshot of COMPLETED_CONFIRMED alone cannot reveal whether the code used a marker, pose-only fallback or gripper-width branch. Recording that branch is necessary for meaningful acceptance evidence.

Offline verification and historical results

The original seven-page AI report states **83 of 84 automated checks**, performed offline with simulated execution, real HOME joint data and a stub model. This number is preserved as a historical report, not rerun coverage of the attached source. Its test scripts, complete assertions and machine logs were not provided.

Table 9. Offline verification and historical results

Historical group	Reported result
Orientation after joint motion, saved return, in-place rotation	19/20; the report attributes one failure to intended refusal of +30° X tilt with the 2 cm collision margin.
Base keep-out with reach enabled and disabled	7/7
Question wording; absolute mode; prompts; comfort threshold	10/10, 4/4 and 13/13
Adjustment versus brute-force comparison	4/4; reported 2–5% distance difference and ≤0.6 s
Corners, timing, dry-run chain and joint ranges	4/4, 5/5 and 8/8
Pseudo-terminal interface	9/9
Total as reported	83/84; offline only, not physical acceptance coverage.

The one reported failure must not be silently converted to a pass. Its description suggests a conflict between a test expectation and an intended refusal, but the missing assertion and trace prevent an independent diagnosis. Moreover, the report's orientation sampling, 90% stretch, independent base keep-out and tour-return behavior do not match the supplied files. Its coverage cannot be assigned to this version without the matching tested source.

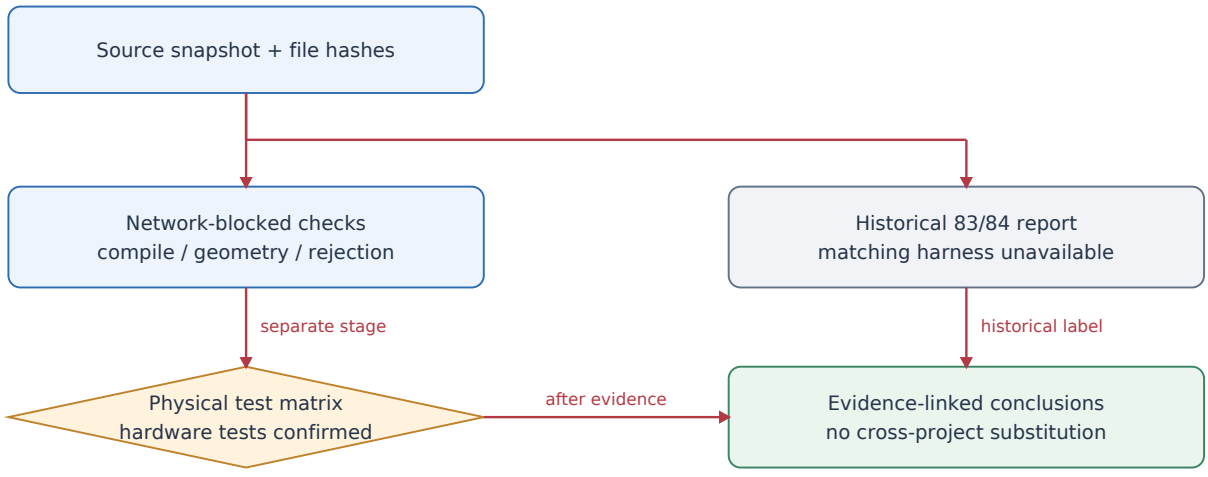


Figure 16. Verification separates executable checks, historical claims and physical measurements.

New checks supplied with this edition

The previous documentation build ran **17/17 focused offline checks** on Python 3.12.14, bypassing CLI startup. These historical checks apply to the earlier source snapshot, not Python 3.13+ compatibility or current-source coverage. Physical testing is separately author-reported using Python 3.14.4 and gpt-oss:20b. No additional hardware results or cross-version/model verification are asserted in v2.1.

Physical UR5e validation

The project author reports that all physical UR5e test scenarios in this section passed. The original implementation was tested using **Python 3.14.4** and **gpt-oss:20b**. Current compatibility is Python 3.13+ with a configurable Ollama model. Other supported versions and models have not all been tested; no new hardware measurements are introduced in this edition.

Hardware and software record

Table 10. Physical UR5e validation

Field	Configuration / supporting record
Robot / controller	UR5e is the intended platform. Serial number and PolyScope version not supplied.
Tool / TCP / payload	OnRobot RG2/RG6 integration in code; exact installed URCap version, TCP calibration and payload record absent.
Computer / runtime	Python 3.13+ supported. Original physical implementation tested with Python 3.14.4 and gpt-oss:20b; other versions/models not all tested.
Workspace / safety setup	Numeric software box and capsule parameters present. Cell layout, fixtures and controller safety configuration not supplied.
Execution record	No per-run initial/final telemetry, timestamped token trace, stop latency or acceptance sign-off supplied.

Physical acceptance matrix

Table 11. Physical UR5e validation

Scenario	Acceptance scope — passed and confirmed
Small +Z motion and saved return	Initial/final TCP and joints; commanded body; result branch; position/orientation errors.
Joint move followed by Cartesian move	Tracked orientation after the joint move; xyz_m uses the updated tool orientation for the Cartesian target.
Arc / circle / polygon	Controller path and blend behavior; end pose; relevant faults and timing.
Reach rejection / adjustment	Requested and accepted target, reason, inspected cell clearance; distinguish software refusal from hardware fault.
/robot and return	Visited/skipped corners, adjusted heights, yaw-following / fixed orientation, direct return or re-verified retracing.
Approval / cancel / timeout	No transmission after cancel or timeout; changed settings/body/state invalidate the task.
Software stop during motion	Delivery on both channels, fresh speeds, stop-confirmation output and independently recorded stop response.
Telemetry or send interruption	Observed controller behavior, reported status and fault latch; do not presume automatic stop.
Gripper width / mismatch / missing register	Command, width samples, sentinel transition, status and visual outcome.

For each row retain date, operator, source SHA-256 manifest, setup versions, initial state, command, expected result, measured result, evidence filename and disposition. Separate planned criteria from observed values. Mark a failure explicitly and preserve its trace before describing a correction or retest.

> All scenarios listed above passed on the physical UR5e, as confirmed by Xamdamov Abu Bakr. These results belong to the AI Motion Planner project; Gesture Control results remain separate.

Limitations, improvements and conclusion

Implemented value

The project provides a substantial inspectable path from language to robot instructions: a constrained planner, deterministic geometric expansion, narrow URScript grammar, numerical path checks, bounded repair, explicit task approval, fresh-state revalidation and differentiated execution results. Local inference keeps planning within the workstation/runtime boundary. Shared validation connects AI-generated tasks and the deterministic corner tour.

Limits established by source review

Model interpretation remains fallible; a numerically valid program can express the wrong object, direction or task. The separate reviewer uses the same model and can share its mistakes. Prompt statements about capabilities are not formal proofs: numerical IK is used internally for validation, while arbitrary invented joint solutions remain prohibited. This reflects a distinction between unsupported free-form joint-goal synthesis and implemented internal reach checks that should be clarified in future prompts.

Nominal DH geometry, finite samples and approximate capsules limit geometric assurance. There is no environment map, obstacle sensor, cable model, payload geometry or full swept-volume test. Rotation-driven samples and post-movej orientation updates are implemented in the current source. The accepted branch in the Python model is not guaranteed to match the controller’s trajectory or configuration.

Approval is strong on the task/tour path but has explicit scope gaps: direct homing has no approval dialog; the settings fingerprint covers only selected fields; translation drift is checked separately from full joint/orientation drift. The SHA-256 body check is content integrity inside the application, not an authenticated or immutable approval protocol.

Telemetry confirmation has fallbacks and conditional fields. The arm wait loop does not stop immediately on loss of telemetry or continuously reject mode changes. Gripper-width confirmation lacks a demonstrated transaction-level timestamp association in the supplied evidence. Local busy locking does not prevent another controller client from interfering.

Prioritized future work

Table 12. Limitations, improvements and conclusion

Priority	Concrete improvement
1 — contract correctness	Unify homing approval policy; broaden fresh joints/orientation comparison and configuration fingerprint coverage.
2 — supervision	Require essential telemetry, associate width readback with a new command, supervise modes continuously and define explicit timeout-stop policy.
3 — geometry	Add rotation-driven sample counts, calibrated kinematics and controller-aware blending; validate geometry against physical measurements.
4 — reproducibility	Attach executable tests to a source revision; capture RTDE/token traces; version physical acceptance evidence and setup metadata.

Conclusion

The implemented system is a useful experimental motion-programming framework with clear separation between language and numeric control. The historical offline result remains 83/84 as reported; the earlier 17-check documentation run is identified separately. The project author confirms that all physical UR5e acceptance tests passed. The strongest next step is to close the documented contract gaps and retain revision-linked records for the confirmed physical robot tests.

Appendix A — configuration reference

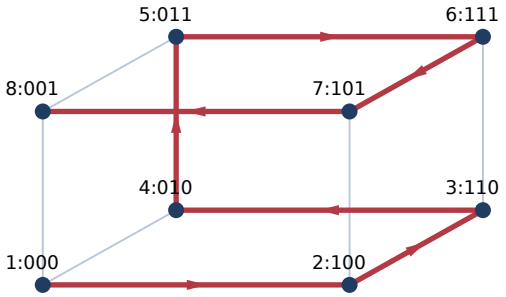
Values below are copied from the attached limits.py and robot_sender.py. Session /limit changes may differ at runtime. These values describe this version, not a recommended universal configuration for another cell.

Table 13. configuration reference

Group	Parameters and values
HOME	[0.49246, −0.135581, 0.260306, 2.217481, 2.218513, −0.007947]; Cartesian metres / rotation-vector radians.
Workspace	Current X [−0.550,0.657]; Y [−0.550,0.550]; Z [0.020180,0.650] m. Earlier: X [0.150,0.657], Y [−0.320,0.320], Z [0.020180,0.500]. Inset 0.001 m.
TCP defaults / caps	Default v=0.02 m/s, a=0.05 m/s²; caps v=0.05, a=0.1. HOME uses 0.05 / 0.1.
Joint defaults / caps	Default v=0.2 rad/s, a=0.3 rad/s²; caps v=0.5, a=1.0.
Joint ranges	Base and wrists [−6,6] rad; shoulder and elbow [−3.3,3.3] rad.
Sampling	Workspace 0.005 m; reach 0.020 m; tour 0.010 m. Joint path 200 intervals.
IK	60 iterations; 0.0005 m / 0.005 rad residuals; numerical Jacobian h=10^{-6}; damping squared 10^{-4}.
Envelope	0.35 rad jump; 0.10 rad rotation step; 0.2 rad singularity margin; stretch 0.90 (earlier 0.95); adjustment buffer 0.02 m.
Capsules	Column r=0.10 m, h=0.30 m; arm 0.08; forearm 0.07; wrist 0.06; tool 0.08; clearance 0.02 m. Independent TCP keep-out: radius 0.20 m below z=0.60 m.
Tour adjustment	Current: shared adjustment, yaw following / fixed option, verified retracing. Historical: 0.05 m buffer, 8 bisections, 0.03 m height gain.
Gripper	Effective width 110−2×2=106 mm; minimum 3 mm; force 3–40 N; default width tolerance 2 mm.
Program budgets	15 plan steps; 40 body lines; repeat ≤20; sleep ≤10 s; initial attempt plus 3 repairs.
Execution	Approval 300 s; state age 1 s; translation drift 0.003 m; arrival 0.002 m / 0.02 rad; timeout 2×estimate+15 s.
Stop	stopj a=2 rad/s²; TCP speed <0.002 m/s; joints <0.005 rad/s; confirmation window 5 s.
Arc	Radius 0.005–0.5 m; sweep ≤350°; first full-circle half blend 0.001 m; blend fraction cap 0.25.

Parameter ownership and reproducibility

limits.py is the principal settings file, but not every operational constant resides there. Tour buffer, circle blending, RPC URL and default width-readback tolerance are defined in other modules. Reproducing a run requires the complete source snapshot, not only a copy of limits.py. The bundle’s SHA-256 manifest identifies each supplied file.



Ideal inset-corner cycle

Red: first seven traversed edges

Gray: box geometry / closing edge

Bits correspond to X, Y, Z

0 = lower face; 1 = upper face

Nearest corner becomes start

Skipped corners alter the path

Figure 17. Geometric Gray-code tour; ideal corner topology, not measured reachability. See Section 14.

Appendix B — source reconciliation

This table records material differences so the redesigned report does not silently combine incompatible revisions. The source remains unchanged. The preserved Figure 2 is explicitly labelled historical because its original artwork must remain recognizable.

Table 14. source reconciliation

Earlier claim	Attached-source finding / report treatment
Python / model	Python 3.13+ gate; original physical tests: 3.14.4 and gpt-oss:20b. Other versions/models not all tested.
Workspace revision	Current X_MIN −0.550, Y ±0.550, Z_MAX 0.650 m. Earlier box preserved in Appendix A.
Stretch / rotation sampling	Current f≤0.90 and ≤0.10 rad rotation steps; earlier f≤0.95 and translation-only sampling are historical.
Independent base keep-out	Always-on TCP cylinder: radius 0.20 m, height 0.60 m; distinct from optional capsule envelope.
Orientation after movej	Current resolver/validator recompute R_BF(q)R_FT; unknown orientation blocks later TCP moves.
base_rotate	movej parameter changes joint 1 via +Y/−Y; omitted rates capped by 0.9 v_max/r and 0.8 a_max/r.
Adjustment / tour	Vertical-half-plane candidate search; tour supports fixed and re-verifies retracing.
Homing / review	Startup and /home still bypass task APPROVE; candidate validation precedes AI review.
Execution confirmation	Marker/rest/pose checks have no-marker fallback; final movej does not compare measured final_q.
Status / evidence	wait=false has no completion fault latch. Historical 83/84 and earlier 17/17 checks remain separate from author-reported hardware tests.

Exact attachment mapping

The project files are main.py, ui.py, ai_agent.py, prompts.py, robot_sender.py, limits.py, tour.py and README.md. These clean names are used consistently throughout the documentation. Full hashes and source-function references accompany the editable report bundle.

> The project’s implementation has not been modified as part of this documentation task. Any future fix requires its own validation and a refreshed report evidence record.

References and traceability

Project sources are authoritative for implementation claims. External references explain interfaces and robotics concepts; they do not establish that this application passed physical acceptance tests. Online references were checked on 3 November 2026.

Project evidence

[P1] Current implementation: UR5e_AI_Motion_Planner_PUBLIC_GitHub.zip, containing main.py, ui.py, ai_agent.py, prompts.py, robot_sender.py, limits.py and tour.py. Earlier source-snapshot checks are historical and are not reassigned to this revision.

[P2] README.md. Deployment assumptions, terminal interface, reported setup experience and project scope.

[P3] UR5e_AI_Agent_Flowcharts.pdf, seven pages. Original page-2 swimlane and historical 83/84 offline verification summary. The associated HTML is preserved in the source bundle.

[P4] UR5e_Gesture_Control_Flowcharts.pdf. Visual reference only: navy/blue palette, information-rich first page, section hierarchy and engineering tables. Its physical results are not AI-planner evidence.

Manufacturer and runtime documentation

[R1] Universal Robots, **Real-Time Data Exchange (RTDE) Guide**. Output fields, protocol and register ranges.
docs.universal-robots.com / tutorials / communication-protocol-tutorials / rtde-guide.html

[R2] Universal Robots, **Primary and Secondary Interfaces**; **Remote Control Via TCP/IP**. Port roles and e-Series local/remote restrictions.
Primary and Secondary Interfaces · Remote Control Via TCP/IP

[R3] Universal Robots, **DH Parameters for Calculations of Kinematics and Dynamics**. Nominal UR5e kinematic dimensions.
universal-robots.com / DH parameters

[R4] Ollama, **Generate a Chat Message**. POST /api/chat request structure and response fields.
docs.ollama.com/api/chat

Implementation traceability

Table 15. References and traceability

Topic	Primary code anchor
Planner / repairs	Agent.build_candidate; ask_json; _blocking_reviewer_errors
Geometry / IK	resolve_plan; ik_solve; ToolModel; reach_walk
Validation / envelope	validate_body; envelope_problem; self_collision_problem
Approval / transport	present_and_execute; Robot.execute; build_urscript
Completion / stop	_wait_for_completion; _finish_gripper; Robot.stop
Tour	build_tour_plan; _highest_reachable_z

General numerical-IK theory is provided here to explain the inspected damped least-squares equations, not to attribute a particular research implementation. Controller-specific behavior and OnRobot integration beyond the cited interfaces remain bounded by the source evidence and the missing physical setup records.